17

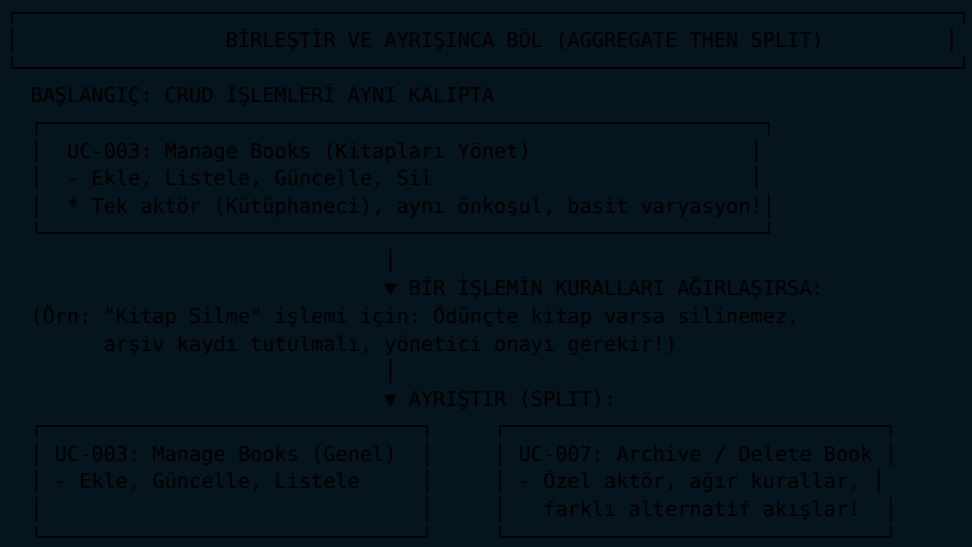
BÖLÜM: TEMEL ESASLAR: AKTÖRLER, DENIZ SEVIYESI HEDEFLER & PAYDAŞ SÖZLEŞMELERI | MODULE: CORE FUNDAMENTALS: ACTORS, SEA-LEVEL GOALS & CONTRACTS

Önce Birleştir, Akışlar Ayrışınca Böl: 'Manage X' Kuralı ve CRUD Birleştirmesi

Aggregate First, Split When Flows Diverge: The 'Manage X' Rule

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir restoranda 'İçecekler' menüsü vardır; çay, kahve, su ve kola hepsi aynı sayfadadır çünkü hepsi bardağa dökülüp servis edilir (Basit CRUD). Ama müşteri 'Özel İsviçre Sıcak Çikolatası' istediğinde, aşçı özel tencere çıkarır, 15 dakika çikolata eritir, içine marshmallow koyar. İşte o zaman o içecek genel menüden çıkarılıp 'Özel Tatlılar' başlığına taşınır! Başta her şeyi bir sepete koy, kuralları karmaşıklaşan şeyi kendi özel kutusuna ayır!

🇬🇧 ENGLISH (ELI5)

On a café menu, standard drinks (tea, drip coffee, water) share a single section because they require identical preparation steps. But when the café introduces traditional siphon-brewed pour-overs involving specialized burners, timing scales, and origin certifications, that beverage earns its own dedicated specialty menu card. Start standard CRUD operations in one consolidated container; split them only when operational rules diverge.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Use Case Sayısı Optimizasyonu ve 'Manage X' Kuralı: Yeni başlayan mimarların en büyük hatası, 4 tabloluk bir veritabanı için 25 tane use case yazmaktır (Create Book, Read Book, Update Book, Delete Book, Search Book…). Bu durum şartname enflasyonuna yol açar ve AIUP'un 'küçük ve okunabilir model' felsefesini yok eder. Cockburn ve Martinelli'nin altın kuralı: 'Aggregate first, split when the flows diverge' (Önce birleştir, akışlar ayrışınca böl). Bir varlığın CRUD işlemleri aynı aktör (Kütüphaneci) tarafından yapılıyor ve temel bir başarı akışını paylaşıyorsa, başlangıçta tek bir 'UC-003: Manage Books' use case'i olarak yazılır. Ancak bu işlemlerden biri (örneğin Block Customer veya Archive Book) kendine has ağır iş kuralları, farklı önkoşullar veya farklı alternatif akışlar geliştirmeye başladığı an, o operasyon derhal 'UC-008: Block Customer' olarak bağımsız bir use case'e dönüştürülür. Bu sayede küçük bir mikroservis 40 değil, sadece 4 ila 8 use case ile kusursuz şekilde modellenir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Managing Use Case Cardinality via Aggregate-then-Split: Novice analysts routinely generate twenty distinct use cases for basic entity management (Create User, Edit User, View User, Delete User, Search User), creating specification bloat that overwhelms engineering reviews. The Martinelli-Cockburn heuristic dictates: start every CRUD resource as a single consolidated 'Manage X' use case. The aggregate holds cleanly as long as the operations share a single primary actor, identical preconditions, and standard data persistence steps. However, the precise moment an operation diverges—e.g. 'Block Account' requiring fraud checks, compliance event logging, and secondary approval flows—it must be excised into an action-level use case. This discipline bounds standard microservices to 4 to 8 use cases total.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: 'Manage X' Çoklu İşlem Yöneticisinin Ayrışma Noktası
package com.maang.aiup.aggregate;

public class ManageBooksService {
    public record Book(String id, String title, String author, boolean isArchived) {}

    // Standart CRUD: Tek bir servis/use case içinde birleşik yaşar
    public void createBook(Book b) { System.out.println("Kitap eklendi: " + b.title()); }
    public void updateBook(Book b) { System.out.println("Kitap güncellendi: " + b.title()); }

    // AYRIŞMA NOKTASI (Ayrı bir Use Case adayı: ArchiveBookUseCase):
    // Ağır iş kuralları: Aktif ödünç kontrolü, denetim kaydı, bildirim!
    public void archiveBook(String bookId, int activeLoanCount) {
        if (activeLoanCount > 0) {
            throw new IllegalStateException("BR-030: Ödünçte kopyası bulunan kitap arşivlenemez!");
        }
        System.out.println("Kitap arşivlendi ve denetim logu üretildi: " + bookId);
    }

    void main() {
        var service = new ManageBooksService();
        service.createBook(new Book("B1", "Clean Code", "Robert Martin", false));
        service.archiveBook("B1", 0);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Bir aktör sisteme asla 'Müşterileri yönetmek' amacıyla oturmaz. Birleştirilmiş 'Manage Customers' use case'i sadece benzer CRUD operasyonlarını bir arada tutan bir başlangıç konteyneridir; kurallar ağırlaştığında bölmekten çekinmeyin.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

An end-user never logs in with the generic goal to 'manage customers'; they come to add or block someone. The 'Manage X' pattern is a structural convenience for uniform CRUD. Split it immediately when business logic diverges.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Başlangıçta CRUD işlemlerini 'Manage X' altında tek bir use case olarak toplayın.
- Bir operasyon kendi özel iş kurallarını ve hata akışlarını geliştirdiğinde hemen ayrı bir use case'e bölün.
- Bu kural sayesinde tipik bir mikroservis 40 dağınık use case yerine 4-8 güçlü use case ile modellenir.

🎯 Key Architectural Takeaways

- Start CRUD operations consolidated under a single 'Manage X' use case.
- Split out an action-level use case the moment preconditions or business rules diverge.
- This aggregation keeps a microservice clean at 4 to 8 comprehensive use cases instead of forty.